Lean 4 w ekonomii matematycznej

Formalizacja twierdzeń i konstrukcja dowodów

Logika, preferencje i optymalizacja

Materiał wprowadzający dla studentów i promotorów

Teoria typów: obiekty i ich rodzaje

Każde poprawne wyrażenie w Lean ma określony typ. Typ wskazuje, jakiego rodzaju obiekt zapisujemy.

Zapis	Znaczenie
`x : ℝ`	Liczba rzeczywista
`v : ℝ × ℝ`	Para liczb rzeczywistych
`f : ℝ → ℝ`	Funkcja o zadanej dziedzinie i przeciwdziedzinie

Zapis `x : ℝ` można na początku czytać podobnie do „x należy do ℝ”. Typ i zbiór są jednak odrębnymi pojęciami w Lean.

Źródło: Theorem Proving in Lean 4, §2.1

Plan seminarium

Czas	Temat
0–35 min	Typy, zapis liczb, zdania i pierwszy dowód
35–55 min	Założenia, teza i podstawowe kroki dowodu
55–75 min	Podstawianie, upraszczanie i rachunki
75–85 min	Przerwa
85–125 min	Użyteczność, preferencje i zbiór budżetowy
125–150 min	Maksymalizacja i ćwiczenia

Praca w parze: przed wpisaniem kolejnej taktyki wyjaśniamy, jakie zadanie dowodowe powinno po niej pozostać.

Wprowadzanie symboli matematycznych

W edytorze VS Code z rozszerzeniem Lean 4 wpisz sekwencję i zatwierdź ją spacją lub Tab.

Symbol	Wpisz	Nazwa w Lean
ℕ	`\N`	`Nat`
ℤ	`\Z`	`Int`
ℚ	`\Q`	`Rat`
ℝ	`\R`	`Real`
ℂ	`\C`	`Complex`

To znaki Unicode używane w kodzie. Na przykład `ℝ` jest notacją typu `Real`. Przykłady z liczbami rzeczywistymi wymagają odpowiedniego importu, tutaj `import Mathlib`.

Polecenie #check: jaki typ ma wyrażenie?

```
#check (3 : ℕ)          -- wynik: 3 : ℕ
#check (3 : ℝ)          -- wynik: 3 : ℝ
#check (2 + 2)          -- wynik: 2 + 2 : ℕ
```

#check odczytuje zapis, uzupełnia informacje o typach i sprawdza, czy wyrażenie jest poprawnie zbudowane.

Następnie pokazuje wyrażenie wraz z jego typem.

Polecenie `#check (2 + 2)` informuje o typie. Do obliczenia wartości wyrażenia służy m.in. `#eval (2 + 2)`, którego wynikiem jest 4.

Zapis 3 w różnych typach liczbowych

```
#check (3 : ℕ)
#check (3 : ℝ)
#check (3 : ℂ)
```

Dopisek po dwukropku wskazuje oczekiwany typ liczby. Lean korzysta z reguły interpretowania zapisu liczbowego w tym typie, opisanej przez `OfNat`.

Gdy w prostym wyrażeniu nie ma innych wskazówek, Lean domyślnie wybiera ℕ.

Ten sam zapis cyfr może oznaczać obiekty różnych typów. Samo to nie ustanawia relacji zawierania pomiędzy tymi typami.

Źródło: Lean Language Reference, Numeric Literals

Obiekt, zdanie i dowód zdania

Zapis	Jak go czytać
`x : ℝ`	x jest liczbą rzeczywistą
`P : Prop`	P jest zdaniem
`h : P`	h jest dowodem zdania P

Równość `2 + 2 = 4` ma typ `Prop`, ponieważ operacja równości tworzy zdanie o dwóch obiektach tego samego typu.

Również `2 + 2 = 5` ma typ `Prop`. Prawdziwość zdania sprawdzamy przez podanie dowodu.

`Prop` klasyfikuje zdania. Dowód konkretnego zdania P ma typ P. Same liczby w `2 + 2 = 4` Lean interpretuje tutaj jako naturalne.

Źródło: Lean Language Reference, Propositions

Odczytywanie deklaracji example

```
example (P : Prop) (hP : P) : P := by
  exact hP
```

Fragment	Znaczenie
`example`	Sprawdź przykład bez nadawania mu nazwy
`(P : Prop)`	Niech P będzie dowolnym zdaniem
`(hP : P)`	Załóżmy, że mamy już dowód P
`: P`	Wymagany wynik: dowód P

To zapis argumentu „jeżeli P, to P”. Nie wynika z niego prawdziwość dowolnego zdania bez założenia jego dowodu.

Zapis := by oraz pierwszy krok dowodu

```
example (P : Prop) (hP : P) : P := by
  exact hP
```

Trzy elementy składni

`: P` określa, co należy dostarczyć.

`:=` rozpoczyna zapis tego, co dostarczamy.

`by` pozwala skonstruować dowód za pomocą taktyk.

Ten sam przykład można zapisać bez taktyk:

```
example (P : Prop) (hP : P) : P := hP
```

Dowód jest już w założeniach. Wystarczy go wskazać.

Dostępne założenia i bieżąca teza

W panelu Infoview przykład ma początkowo postać:

```
P : Prop
hP : P
⊢ P
```

Nad znakiem ⊢ widzimy obiekty i założenia dostępne w danym miejscu dowodu.

Za znakiem ⊢ widzimy zdanie, którego dowodu Lean teraz oczekuje.

Dokumentacja nazywa listę dostępnych obiektów i założeń „kontekstem lokalnym". W prezentacji posługujemy się opisem „dostępne założenia".

exact: wskazanie kompletnego dowodu

Przed krokiem

```
P : Prop
hP : P
⊢ P
```

Krok dowodowy

```
exact hP
```

Dowód `hP` ma typ `P`, czyli odpowiada bieżącej tezie.

Po tym kroku nie pozostaje żadne zadanie dowodowe.

`exact` może wskazać także dowód złożony z kilku dostępnych faktów, np. `exact hPQ hP` stosuje dowód implikacji do dowodu jej przesłanki.

intro: przyjęcie przesłanki implikacji

```
example (P Q : Prop) (hPQ : P → Q) : P → Q := by
  intro hP
  exact hPQ hP
```

Przed `intro hP`

```
P Q : Prop
hPQ : P → Q
⊢ P → Q
```

Po `intro hP`

```
P Q : Prop
hPQ : P → Q
hP : P
⊢ Q
```

Aby dowieść implikacji, przyjmujemy jej przesłankę i dowodzimy wniosku. Przy tezie `∀ x, ...` taktyka `intro x` wprowadza dowolny obiekt x.

apply: sprowadzenie tezy do przesłanek

Dysponujemy twierdzeniem o przechodniości: z $x \leq y$ i $y \leq z$ wynika $x \leq z$.

```
example (x y z : ℝ)
    (hxy : x ≤ y) (hyz : y ≤ z) : x ≤ z := by
  apply le_trans hxy
  exact hyz
```

Etap	Pozostaje udowodnić
Przed `apply`	$x \leq z$
Po `apply le_trans hxy`	$y \leq z$
Po `exact hyz`	Brak dalszych zadań

`apply` dobiera twierdzenie do bieżącej tezy. Nieuzupełnione przesłanki stają się nowymi zadaniami dowodowymi.

Koniunkcja: dowód i analiza hipotezy

Teza `P ∧ Q`

```
example (P Q : Prop)
    (hP : P) (hQ : Q) :
    P ∧ Q := by
  constructor
  · exact hP
  · exact hQ
```

Aby dowieść koniunkcji, dowodzimy obu zdań. `constructor` rozdziela te zadania.

Hipoteza `h : P ∧ Q`

```
example (P Q : Prop)
    (h : P ∧ Q) : P := by
  obtain ⟨hP, hQ⟩ := h
  exact hP
```

Z założenia otrzymujemy osobno dowód P i dowód Q.

Zapis `rcases h with ⟨hP, hQ⟩` również wyodrębnia oba dowody. Kropka `·` otwiera osobny poddowód.

Istnienie: świadek i jego własność

Dowód istnienia

```
example : ∃ x : ℝ,
    x > 0 := by
  use 1
  norm_num
```

`use 1` wybiera świadka. Taktyka `norm_num` uzasadnia nierówność $1 > 0$.

Użycie faktu o istnieniu

```
example (P : ℝ → Prop)
    (h : ∃ x, P x) :
    ∃ y, P y := by
  obtain ⟨x, hx⟩ := h
  use x
```

Świadek: x. Jego własność: `hx : P x`.

W pierwszym przypadku wybieramy świadka tezy. W drugim korzystamy ze świadka dostępnego w założeniu.

Ćwiczenia 1 i 3: założenia i koniunkcja

Zadanie 1

```
example (P : Prop) (hP : P) : P := by
  sorry
```

Zadanie 3

```
example (P Q : Prop) (h : P ∧ Q) : Q ∧ P := by
  sorry
```

Przed wpisaniem kodu: jakie dowody są już dostępne i jakie cele powinien utworzyć pierwszy krok?

rw: podstawienie zgodnie z równością

```
example (x y : ℝ) (h : x = y) : x + 1 = y + 1 := by
  rw [h]
```

Założenie `h : x = y` pozwala zastąpić x przez y. W tym przykładzie otrzymujemy identyczne wyrażenia po obu stronach, więc `rw` kończy dowód.

Zapis	Miejsce i kierunek podstawienia
`rw [h]`	W tezie: x zastępujemy przez y
`rw [← h]`	W tezie: y zastępujemy przez x
`rw [h] at hx`	W założeniu hx: x zastępujemy przez y

simp: uproszczenie do znanej postaci

```
def twice (x : ℝ) : ℝ := 2 * x

example (x : ℝ) : twice x + 0 = 2 * x := by
  simp [twice]
```

`simp [twice]` rozwija podaną definicję i stosuje reguły upraszczania, np. $a + 0 = a$.

W tym przykładzie obie strony upraszczają się do $2x$, co kończy dowód.

Reguły upraszczania są uzasadnionymi twierdzeniami bibliotecznymi. Taktyka `simp` może uprościć zadanie bez jego pełnego rozwiązania.

rfl: równość wynikająca z definicji

```
def twice (x : ℝ) : ℝ := 2 * x

example (x : ℝ) : twice x = 2 * x := by
  rfl
```

Po rozwinięciu definicji `twice` obie strony są tym samym wyrażeniem.

`rfl` wystarcza, gdy Lean rozpoznaje taką zgodność przez dozwolone obliczenia i redukcję definicji.

Przemienność `x + y = y + x` dla dowolnych x i y wymaga twierdzenia o dodawaniu. Sama redukcja definicji jej tutaj nie uzasadnia.

Tożsamości wielomianowe

```
example (x y : ℝ) (h : x = y + 1) :
    x ^ 2 = y ^ 2 + 2 * y + 1 := by
  rw [h]
  ring
```

Podstawienie i normalizacja

`rw [h]` zastępuje x przez y+1 zgodnie z założeniem.

`ring` sprowadza obie strony do tej samej postaci wielomianowej.

Normalizacja wielomianów nie zastępuje świadomego doboru podstawienia.

Nierówności liniowe

```
example (x y : ℝ)
    (hx : 2 * x + y ≤ 7) (hy : 1 ≤ y) :
    x ≤ 3 := by
  linarith
```

Rozumowanie matematyczne

$$2x + 1 \leq 2x + y \leq 7 \quad \Longrightarrow \quad x \leq 3.$$

`linarith` uzasadnia wniosek przez połączenie dostępnych równości i nierówności liniowych.

Przy mnożeniu przez parametr trzeba uwzględnić jego znak, np. przez `mul_le_mul_of_nonneg_left`.

Nieujemność kwadratu w dowodzie nierówności

Teza

$$\forall x, y \in \mathbb{R} : \quad 2xy \le x^2 + y^2.$$

```
example (x y : ℝ) : 2 * x * y ≤ x ^ 2 + y ^ 2 := by
  have hsq : 0 ≤ (x - y) ^ 2 := sq_nonneg (x - y)
  nlinarith [hsq]
```

`have` dodaje dowód nieujemności kwadratu. `nlinarith` wyprowadza z tego faktu żądaną nierówność.

Uzasadnienie przesłanki `hsq`:

$$(x - y)^2 = x^2 - 2xy + y^2 \ge 0.$$

Ćwiczenie 7: dobierz kwadrat uzasadniający $2x \le x^2 + 1$. Taktyka `nlinarith` może wymagać takiej dodatkowej przesłanki.

Jawne założenia modelu

```
example {X : Type*} (R : X → X → Prop)
    (hTrans : ∀ a b c, R a b → R b c → R a c)
    (x y z : X) (hxy : R x y) (hyz : R y z) :
    R x z := by
  exact hTrans x y z hxy hyz
```

- `X` jest typem alternatyw, a `R` relacją na `X`.
- `hTrans` jest jawnym założeniem o tej relacji.
- Wynik wyprowadza konsekwencję przyjętej własności.

Globalne `axiom` przyjmuje deklarację bez dowodu. Założenia ekonomiczne zachowujemy jawnie w treści modelu i twierdzeń.

Monotoniczność użyteczności afinicznej

Twierdzenie

Niech $u(x) = ax + b$ oraz $a \geq 0$. Wówczas

$$x \leq y \quad \Longrightarrow \quad u(x) \leq u(y).$$

Dowód na papierze:

$$x \leq y \quad \Longrightarrow \quad ax \leq ay \quad \Longrightarrow \quad ax + b \leq ay + b.$$

- Pierwsze przejście wykorzystuje nieujemność a.
- Drugie wykorzystuje zgodność porządku z dodawaniem.

`Monotone` oznacza niemalejącość. Ścisłą rosnącość zapisujemy jako `StrictMono`.

Dowód monotoniczności w Lean

```
theorem affine_monotone (a b : ℝ) (ha : 0 ≤ a) :
    Monotone (fun x : ℝ => a * x + b) := by
  intro x y hxy
  change a * x + b ≤ a * y + b
  have hmul : a * x ≤ a * y :=
    mul_le_mul_of_nonneg_left hxy ha
  exact add_le_add hmul (le_refl b)
```

`intro` wprowadza argumenty i przesłankę nierówności. `change` pokazuje wartości funkcji. `have` zapisuje krok mnożenia, a `exact` zamyka dowód.

Znak parametru a monotoniczność

$$u(x) = ax + b, \quad x \in \mathbb{R}.$$

Parametr	Własność funkcji	Zapis w Mathlib
$a > 0$	Ściśle rosnąca	`StrictMono u`
$a = 0$	Stała, niemalejąca	`Monotone u`
$a < 0$	Ściśle malejąca	`StrictAnti u`

Ćwiczenie 8. Udowodnij właśność ściśle rosnącą dla $a > 0$.

Ćwiczenie 9. Dla $u(x) = -x$ wskaż kontrprzykład do niemalejącości.

Dodatniość współczynnika pozwala zachować ścisłą nierówność przy mnożeniu.

Preferencje reprezentowane przez użyteczność

Definicja

Dla $u : X \to \mathbb{R}$ przyjmujemy

$$x \succeq y \quad \Longleftrightarrow \quad u(y) \leq u(x).$$

```
def weakPref {X : Type*} (u : X → ℝ)
    (x y : X) : Prop := u y ≤ u x
```

`weakPref u x y` czytamy: x jest co najmniej tak samo pożądany jak y.

Źródło: Osborne i Rubinstein (2023), rozdz. 1

Przechodniość preferencji

$$u(z) \le u(y) \le u(x) \quad \Longrightarrow \quad x \succeq z.$$

```
theorem weakPref_trans {X : Type*} (u : X → ℝ)
    (x y z : X) (hxy : weakPref u x y)
    (hyz : weakPref u y z) : weakPref u x z := by
  change u y ≤ u x at hxy
  change u z ≤ u y at hyz
  change u z ≤ u x
  exact le_trans hyz hxy
```

Po rozwinięciu definicji korzystamy z przechodniości porządku w $\mathbb{R}$. Kolejność argumentów `hyz`, `hxy` wynika z kierunku nierówności.

Zwrotność, zupełność i zakres wyniku

Własność	Po rozwinięciu definicji	Uzasadnienie
Zwrotność	$u(x) \leq u(x)$	`le_refl (u x)`
Zupełność	$u(y) \leq u(x) \vee u(x) \leq u(y)$	`le_total (u y) (u x)`

Znaczenie założonej reprezentacji

Relacja zdefiniowana za pomocą użyteczności jest zupełna i przechodnia. Istnienie reprezentacji dla niezależnie zadanej relacji wymaga osobnego twierdzenia.

Obojętność nie implikuje równości alternatyw. Różne obiekty mogą mieć tę samą użyteczność.

Porządkowy charakter użyteczności

Jeżeli $\varphi : \mathbb{R} \to \mathbb{R}$ jest ściśle rosnąca, to

$$u(y) \leq u(x) \quad \Longleftrightarrow \quad \varphi(u(y)) \leq \varphi(u(x)).$$

Funkcje u i $\varphi \circ u$ reprezentują tę samą relację preferencji.

Przykład

$$v(x) = 3u(x) + 7.$$

Zmiana skali zachowuje uporządkowanie alternatyw.

Różnice wartości użyteczności nie opisują tutaj intensywności preferencji. Formalny dowód niezmienniczości jest propozycją dalszej pracy.

Źródło: Osborne i Rubinstein (2023), §1.4

Zbiór budżetowy dwóch dóbr

$$B(p, w) = \{x \in \mathbb{R}^2 : x_1 \geq 0, x_2 \geq 0, p_1 x_1 + p_2 x_2 \leq w\}.$$

```
abbrev Bundle := ℝ × ℝ

def spending (p₁ p₂ : ℝ) (x : Bundle) : ℝ :=
  p₁ * x.1 + p₂ * x.2

def budgetSet (p₁ p₂ w : ℝ) : Set Bundle :=
  {x | 0 ≤ x.1 ∧ 0 ≤ x.2 ∧ spending p₁ p₂ x ≤ w}
```

Parametr w oznacza dostępny budżet. Składowe koszyka zapisujemy jako `x.1` i `x.2`.

Zwiększenie budżetu i inkluzja zbiorów

Twierdzenie

$$w_1 \le w_2 \implies B(p, w_1) \subseteq B(p, w_2).$$

```
theorem budget_mono_income (p₁ p₂ w₁ w₂ : ℝ)
    (hw : w₁ ≤ w₂) :
    budgetSet p₁ p₂ w₁ ⊆ budgetSet p₁ p₂ w₂ := by
  intro x hx
  obtain ⟨hx1, hx2, hxw⟩ := hx
  exact ⟨hx1, hx2, le_trans hxw hw⟩
```

Ilości dóbr pozostają nieujemne. Nowy limit wydatków wynika z przechodniości: wydatek ≤ w_1 ≤ w_2.

Wypukłość zbioru budżetowego

Definicja wypukłości

Dla $x, y \in B$ oraz $a, b \geq 0$ takich, że $a + b = 1$,

$$ax + by \in B.$$

Nieujemność składowych kombinacji wynika z nieujemności wag i składowych koszyków.

Dla wydatków:

$$p \cdot (ax + by) = a(p \cdot x) + b(p \cdot y) \leq aw + bw = (a + b)w = w.$$

W Lean korzystamy ze standardowej definicji `Convex ℝ`, zgodnej z tą kombinacją wypukłą.

Wypukłość: warunki dopuszczalności

Niech $x, y \in B(p, w)$, $a, b \geq 0$ oraz $a + b = 1$. Dla $z = ax + by$ dowodzimy:

Warunek	Uzasadnienie w Lean
$z_1 = ax_1 + by_1 \geq 0$	`positivity`
$z_2 = ax_2 + by_2 \geq 0$	`positivity`
$p \cdot z \leq w$	`calc` i twierdzenia o porządku

Kluczowe przejście dla wydatków:

$$a(p \cdot x) + b(p \cdot y) \leq aw + bw = w.$$

Oszacowanie wykorzystuje nieujemność wag. Ostatnia równość korzysta z warunku a+b=1. Pełny dowód: część G skryptu.

Założenia a własności zbioru budżetowego

Własność	Założenia wystarczające
Wypukłość	Dowolne $p_1, p_2, w \in \mathbb{R}$
Niepustość	$w \geq 0$, świadek $(0, 0)$
Ograniczoność	$p_1 > 0$, $p_2 > 0$ oraz $w \geq 0$

Ćwiczenie 11. Uzasadnij niepustość przez wskazanie koszyka zerowego.

Przy dodatnich cenach każdy dopuszczalny koszyk spełnia

$$0 \leq x_i \leq \frac{w}{p_i} \quad (i = 1, 2).$$

Ograniczoność omawiamy matematycznie. Skrypt formalizuje wypukłość, a ćwiczenie 11 dotyczy niepustości.

Źródło: Osborne i Rubinstein (2023), §5.1; zakres formalnego lematu jest szerszy

Globalne maksimum funkcji kwadratowej

Twierdzenie

Dla $u_A(x) = Ax - x^2$, gdzie $A, x \in \mathbb{R}$,

$$u_A(x) \leq u_A\left(\frac{A}{2}\right).$$

Certyfikat optymalności:

$$u_A\left(\frac{A}{2}\right) - u_A(x) = \left(x - \frac{A}{2}\right)^2 \geq 0.$$

Oszacowanie obowiązuje na całym $\mathbb{R}$, dla każdego parametru A.

Dowód optymalności w Lean

```
def quadraticUtility (A x : ℝ) : ℝ := A * x - x ^ 2

theorem quadratic_global_max (A x : ℝ) :
    quadraticUtility A x ≤ quadraticUtility A (A / 2) := by
  dsimp [quadraticUtility]
  have hsq : 0 ≤ (x - A / 2) ^ 2 := sq_nonneg (x - A / 2)
  nlinarith [hsq]
```

- `dsimp` rozwija definicję funkcji celu.
- `have` dodaje uzasadniony fakt pomocniczy.
- `nlinarith` weryfikuje rachunek wielomianowy.

Maksymalizacja na półprostej x ≥ 0

```
theorem quadratic_nonneg_optimum (A : ℝ) (hA : 0 ≤ A) :
    ∃ x : ℝ, 0 ≤ x ∧ ∀ y : ℝ, 0 ≤ y →
      quadraticUtility A y ≤ quadraticUtility A x := by
  use A / 2
  constructor
  · positivity
  · intro y _hy
    exact quadratic_global_max A y
```

Rola założenia A ≥ 0

Zapewnia dopuszczalność kandydata $\frac{A}{2}$. Optymalność wynika z wcześniejszego oszacowania na całym $\mathbb{R}$.

Ćwiczenie 10: górne oszacowanie funkcji

Zadanie 10

```
example (x : ℝ) : 6 * x - x ^ 2 ≤ 9 := by
  sorry
```

1. Wskaż punkt osiągania wartości 9.
2. Zapisz różnicę między 9 a wartością funkcji jako kwadrat.
3. Zastosuj `sq_nonneg` i odpowiednią taktykę arytmetyczną.

Dalszy projekt: maksimum Ax − x² na x ≥ 0 dla dowolnego A. Wymaga rozpatrzenia znaku parametru.

Punkt maksymalizujący i wartość maksymalna

Dla $u_A(x) = Ax - x^2$ na całym $\mathbb{R}$:

Wielkość	Wynik
Punkt maksymalizujący	$x^* = \frac{A}{2}$
Wartość maksymalna	$u_A(x^*) = \frac{A^2}{4}$

Na dziedzinie $x \geq 0$ punkt maksymalizujący wynosi

$$x^* = \max\left\{\frac{A}{2}, 0\right\}.$$

Skrypt dowodzi istnienia rozwiązania dla $A \geq 0$. Przypadek $A < 0$, w którym rozwiązaniem jest $x = 0$, pozostaje zadaniem rozszerzającym.

Kontrola treści formalnego wyniku

- Czy dziedziny i kierunki relacji odpowiadają modelowi?
- Gdzie wykorzystujemy dodatniość, niezerowość i dopuszczalność?
- Czy założenia nie czynią tezy niezamierzenie trywialną?
- Czy wynik korzysta wyłącznie z zaakceptowanych aksjomatów?

```
#print axioms quadratic_nonneg_optimum
```

W tym przykładzie otrzymujemy standardowe zależności `propext`, `Classical.choice`, `Quot.sound`. Obecność `sorryAx` wskazywałaby na lukę.

Samodzielna formalizacja lematu

Punkt wyjścia kolejnego spotkania

Twierdzenie z własnego modelu, którego dowód na papierze zajmuje kilkanaście wierszy, wraz z definicjami i pełną listą założeń.

1. Dobór definicji odpowiadających interfejsom Mathlib.
2. Podział dowodu na małe lematy.
3. Wyszukanie twierdzeń i kontrola ich typów przez `#check`.
4. Formalny dowód oraz ponowna kontrola jego interpretacji.

Możliwy temat: zachowanie preferencji przez ściśle rosnącą transformację użyteczności.

Liczby w innych strukturach

```
#check (3 : ℂ)
#check (3 : Quaternion ℝ)
#check (3 : Matrix (Fin 2) (Fin 2) ℝ)
```

W tych strukturach 3 oznacza odpowiednio liczbę zespoloną, kwaternion i macierz skalarną.

W standardowej strukturze macierzy kwadratowych zapis 3 oznacza $3I$, z zerami poza przekątną.

Macierz samych trójek to inny obiekt: `fun (_ _ : Fin 2) => (3 : ℝ)`. Dla kwaternionów zapis `ℍ[ℝ]` wymaga `open scoped Quaternion`. Przykłady zakładają `import Mathlib`.

Zapis liczbowy i przekształcenie typu

Interpretacja zapisu 3

```
#check (3 : ℝ)
```

`OfNat` określa, jaki obiekt typu $\mathbb{R}$ oznacza ten zapis cyfr.

Przekształcenie zmiennej

```
example (n : ℕ) : ℝ := n
```

Lean wstawia dostępne przekształcenie z $\mathbb{N}$ do $\mathbb{R}$.

Typy $\mathbb{N}$ i $\mathbb{R}$ pozostają odrębne. Biblioteka opisuje przekształcenia między nimi oraz struktury algebraiczne.

Źródło: Lean Language Reference, Coercions

Literatura: formalizacja w Lean

Theorem Proving in Lean 4

Rozdział 3, Propositions and Proofs
Zdania jako typy i konstrukcja termów dowodowych.

Mathematics in Lean

Rozdział 2, Basics i rozdział 3, Logic
Równości, struktury algebraiczne i logika.

Dokumentacja Mathlib i Lean Language Reference

Taktyki Mathlib oraz zależności aksjomatyczne.

Działanie kodu zależy od wersji projektu. Pełna bibliografia i odsyłacze do definicji znajdują się w przewodniku.

Literatura: podstawy mikroekonomiczne

Martin J. Osborne, Ariel Rubinstein

Models in Microeconomic Theory
Expanded Second Edition, Open Book Publishers, 2023

Fragment	Temat w seminarium
Rozdział 1, s. 3–16	Preferencje i reprezentacja użytecznościowa
§1.4	Porządkowy charakter użyteczności
§5.1, s. 57–58	Zbiór budżetowy i jego wypukłość

doi.org/10.11647/OBP.0362

Przykłady w Lean stanowią opracowanie seminaryjne. Lemat o wypukłości ma słabsze założenia niż podręcznikowy model konsumenta.